NAME : PRIYADHARSHINI K

REG NO : 192421236

COURSE : CSA0927 – PROGRAMMING IN JAVA

Q1 **Debug Nested Exception Handling**

```
import java.util.Scanner;

public class Debug1 {
public static void main(String args[]) {
Scanner sc = new Scanner(System.in);

int arr[] = {20,40,60,80};

int index = sc.nextInt();
int divisor = sc.nextInt();

try {
int value = arr[index];
try {
int result = value/divisor;
System.out.println(result);
}
catch(ArrayIndexOutOfBoundsException e) {
System.out.println("Invalid Index");
}
}
catch(ArithmeticException e) {
System.out.println("Division by Zero");
}
}
}
```

Debugging Task

Identify why some exceptions are never caught.

Modify the exception handling so all runtime exceptions are handled correctly.

TEST CASE 1

Input 2 5

Expected Output: 12

TEST CASE 2

Input 5 2

Expected Output : Invalid Index

TEST CASE 3

Input 1 0

Expected Output : Division by Zero

Your Fixed Code

```java
import java.util.Scanner;

public class Debug1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int arr[] = {20, 40, 60, 80};

        int index = sc.nextInt();
        int divisor = sc.nextInt();

        try {
            int value = arr[index];
            int result = value / divisor;
            System.out.println(result);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid Index");
        } catch (ArithmeticException e) {
            System.out.println("Division by Zero");
        }
    }
}
```

Test Input

```
1 2
5 2
1 0
```

Output

```
20
```

Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Q2 Debug Exception Propagation

```java
import java.util.Scanner;

class Calculator{

static int divide(int a,int b){
return a/b;
}

static void display(int x){
System.out.println("Answer = "+x);
}

}

public class Debug2{

public static void main(String args[]){

Scanner sc=new Scanner(System.in);

int a=sc.nextInt();
int b=sc.nextInt();

Calculator.display(Calculator.divide(a,b));

System.out.println("Execution Completed");
```

}

}

Debugging Task

Prevent abnormal termination.

Ensure "Execution Completed" is always printed.

TEST CASE 1

INPUT: 20 5

OUTPUT : Answer = 4

Execution Completed

TEST CASE 2

INPUT: 50 0

OUTPUT: Cannot Divide by Zero

Execution Completed

TEST CASE 3

INPUT: 36 6

OUTPUT: Answer = 6

Execution Completed

Your Fixed Code

```java
import java.util.Scanner;
public class Calculator {
    static int divide(int a, int b) {
        return a / b;
    }
    static void display(int x) {
        System.out.println("Answer = " + x);
    }
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();

        try {
            display(divide(a, b));
        } catch (ArithmeticException e) {
            System.out.println("Cannot Divide by Zero");
        } finally {
            System.out.println("Execution Completed");
        }
    }
}
```

Test Input

```
20 5
50 0
36 6
```

Output

```
Answer = 4
Execution Completed
```

Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Q3 Debug Multiple Runtime Exceptions

```
import java.util.Scanner;
public class Debug3 {
public static void main(String args[]) {
Scanner sc=new Scanner(System.in);
int arr[]={12,24,36,48};
String number=sc.next();
int index=Integer.parseInt(number);
System.out.println(arr[index]);
}
}
```

Debugging Task

The program may generate two different runtime exceptions.

Modify the program so that each exception is handled separately.

Test Case 1
INPUT: 2
OUTPUT: 36
Test Case 2
INPUT: ABC
OUTPUT: Invalid Number
Test Case 3
INPUT: 5
OUTPUT: Invalid Array Index

Your Fixed Code

```java
import java.util.Scanner;
public class Debug3 {
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        int arr[] = {12, 24, 36, 48};
        String number = sc.next();
        try {
            int index = Integer.parseInt(number);
            try {
                System.out.println(arr[index]);
            }
            catch (ArrayIndexOutOfBoundsException e){
                System.out.println("Invalid Array Index");
            }
        }
        catch (NumberFormatException e){
            System.out.println("Invalid Number");
        }
    }
}
```

Test Input

```
2
ABC
5
```

Output

```
36
```

Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Q4 Debug Finally Execution

```
public class Debug4 {
static int calculate(int a,int b){
try{
return a/b;
}
finally{
System.out.println("Resources Released");
}
}
public static void main(String args[]){
System.out.println(calculate(20,0));
}
}
```

Debugging Task
Prevent program termination.
Ensure the finally block executes while handling the exception correctly.
Test Case 1
INPUT: 20 5
OUTPUT: Resources Released
4
Test Case 2
20 0
Resources Released
Division by Zero
Test Case 3
81 9
Resources Released
9

Your Fixed Code

```java
import java.util.Scanner;

public class Debug4 {

    static int calculate(int a, int b) throws ArithmeticException {
        try {
            return a / b;
        } finally {
            System.out.println("Resources Released");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int a = sc.nextInt();
        int b = sc.nextInt();

        try {
            System.out.println(calculate(a, b));
        } catch (ArithmeticException e) {
            System.out.println("Division by Zero");
        }
    }
}
```

Test Input

```
20 5
20 0
81 9
```

Output

```
Resources Released
4
```

Test Cases

- Test Case 1 ✓
- Test Case 2 ✓
- Test Case 3 ✓

10.00 / 10 marks

Q5 Debug the NullPointerException

public class Debug4 {
public static void main(String[] args) {
String name = null;
System.out.println(name.length());
}
}
Expected Task
Prevent the program from crashing.
Test Case 1
INPUT: Java
OUTPUT: Length = 4
Test Case 2
INPUT: null
OUTPUT: String is Null
Test Case 3
INPUT: Programming
OUTPUT: Length = 11

Your Fixed Code

```java
import java.util.Scanner;

public class Debug4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Check if there is input available
        if (sc.hasNextLine()) {
            String name = sc.nextLine();

            // Test Case 2 logic: check if the input string itself is 
            // or if the variable is null
            if (name.equals("null") || name == null) {
                System.out.println("String is Null");
            } else {
                System.out.println("Length = " + name.length());
            }
        }
        sc.close();
    }
}
```

Test Input

```
java
null
Programming
```

Output

```
Length = 4
```

Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks